Conception Détaillée EvolutionRobot

Défintions des types et constantes

Booléen redresserADroite ← (**digitalRead** ($LINE_SENSOR_RIGH$) = 1) **and**
(**digitalRead** ($LINE_SENSOR_FRONT$) = 0)
Booléen redresserAGauche ← (**digitalRead** ($LINE_SENSOR_LEFT$) = 1) **and**
(**digitalRead** ($LINE_SENSOR_FRONT$) = 0)
Booléen intersection ← [(**digitalRead** ($LINE_SENSOR_LEFT$) = 1) **and**
(**digitalRead** ($LINE_SENSOR_FRONT$) = 1)] **ou**
[(**digitalRead** ($LINE_SENSOR_RIGHT$) = 1) **and**
(**digitalRead** ($LINE_SENSOR_FRONT$) = 1)]
Enumération Ordre = {AV, TD, TG, .}
Enumération EtatDAvancement = {EAvancer, EIntersection, EUrgence, ESorti}

Procédures

procedure avancer (**E/S :** etat : EtatDAvancement)
début

```
tant que (non instersection) et (etat ≠ EUrgence) faire
    si ROBOT_arretDUrgence () alors
        etat ← EUrgence
    sinon
        ROBOT_avancer (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4, PWM_EN1,
         PWM_EN2)
        si redrresserADroite alors
            ROBOT_redresser (PWM_EN2, PWM_EN1)
            répéter
            jusqu'à digitalRead (LINE_SENSOR_FRONT) = 1;
            ROBOT_redresser (PWM_EN1, PWM_EN2)
            attendre ()
            ROBOT_avancer (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4, PWM_EN1,
             PWM_EN2)
        sinon si redrresserAGauche alors
            ROBOT_redresser (PWM_EN1, PWM_EN2)
            répéter
            jusqu'à digitalRead (LINE_SENSOR_FRONT) = 1;
            ROBOT_redresser (PWM_EN2, PWM_EN1)
            attendre ()
            ROBOT_avancer (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3, MOTEUR_IN4, PWM_EN1,
             PWM_EN2)
        fin
        etat ← EIntersection
    fin
fin
```

fin

```
procedure tourner (E/S : etat : EtatDAvancement, E : prochaineAction : Ordre)
début
   suivant prochainVirage faire
      cas où TD faire
         ROBOT_tournerDroite (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3,
          MOTEUR_IN4)
         attendre()
         etat ← EAvancer
      cas où TG faire
         ROBOT_tournerGauche (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3,
          MOTEUR_IN4)
         attendre()
         etat ← EAvancer
      cas où AV faire
         etat ← EAvancer
      autres cas faire
         etat = ESorti
      fin
   fin
fin

procedure evolutionRobot (E/S : etat : EtatDAvancement, E : listeDOrdres : Liste <Ordre>)
    Déclaration :  prochaineAction : Ordre
                   etat : EtatDAvancement
                   estSorti : Booléen
                   numOrdre : Naturel Non Nul
début
   numOrdre ← 1
   etat ← avancer
   prochaineAction ← listeOrdre[numOrdre]
   tant que etat ≠ ESorti et prochaineAction ≠ . faire
      suivant etat faire
         cas où EAvancer faire
            avancer (etat)
         fin
         cas où EIntersection faire
            tourner (etat)
         fin
         cas où EUrgence faire
            ROBOT_arreter (MOTEUR_IN1, MOTEUR_IN2, MOTEUR_IN3,
             MOTEUR_IN4)
         fin
      fin
      numOrdre ← numOrdre + 1
      prochaineAction ← listeOrdre[numOrdre]
   fin
fin
```